bjt-agent 技术架构与 Agent 工程指导

> **面向 AI Agent 工程师的系统教程。**基于 2026-10 生产代码现状编写（`bjt_agent` 仓库，sales 工作树 `E:\dvlp\dev\sales\bjt-agent`，`master-agent-sales` 分支）。 约定：本文只写架构、协议与 IP，**不写任何密码/密钥**（凭证见 sales 工作树 `doc/workspace/05-production-env.md`）。文中代码引用格式为 `相对路径（函数/类名）`，均相对 `bjt-agent/` 仓库根。

目录

1. 系统概览
2. 总体架构
3. 生产部署拓扑
4. 核心框架层：Mini-Agent
5. 领域 Agent 层：六种形态
6. 工具层设计
7. LLM 调用与用量治理
8. 任务工程与可靠性
9. 数据架构
10. 核心工作流程走读
11. 经验自学习闭环
12. Agent 工程模式清单（事故沉淀）
13. 扩展开发指南
14. 附录：术语表与文档索引

---

1. 系统概览

1.1 产品形态

bjt-agent（标捷通 · AI 标书审查平台）是一个面向投标业务的 **AI Agent SaaS**：用户上传招标文件与投标文件，平台用 LLM 智能体逐规则审查投标文件的符合性、检查格式违规（暗标）、比对多份标书的雷同度（查重），并反向提供 AI 编标（生成投标文件）能力。入口 `check.aibjt.com:30002`（公网 NAT 到 prod nginx :8000）。

系统有两个使用终端：

- **Web 前端**（Vue3 SPA）：项目/文档管理、发起审查、实时查看执行时间线、结果与报告；

- **Word 插件（VSTO）**：用户在自己电脑的 Word 里装插件，平台 Agent 通过"远程工具协议"**反向调用用户 Word 进程**执行文档级检查（暗标检查的核心通道），插件经 WebView2 内嵌平台页面完成 SSO。

登录体系外包给运营平台（operate-two，Java）：`POST /api/auth/login` 实际转发运营台 `/aiCheckLogin` 校验，成功后本地 find-or-create 用户并签发 JWT（claims 含 `interior_user` 内部员工标记与 `concurrency` 并发配额）。这决定了 bjt-agent 没有独立的账号密码体系。

1.2 业务能力清单

能力	入口	负责 Agent	形态
标书审查（多规则并行）	`/api/projects/{id}/review`	MasterAgent + BidReviewAgent	Master-SubAgent 编排 + ReAct
技术标查重（多文档比对）	`/api/projects/{id}/duplicate-check`	DuplicateMasterAgent + DuplicateCheckAgent	编排 + 确定性检索 + 单次 LLM 判定
暗标符合性检查（格式合规）	`/api/blind-check` + VSTO 插件	BlindCheckAgent	确定性优先 + 受限 ReAct
AI 标书生成（V1）	`/api/bid-draft`	BidDraftAgent	纯代码编排 LLM 管线
AI 编标（V2，素材+问卷+逐章）	`/api/bid-wizard`	bid_wizard_agent 系列	纯代码编排 LLM 管线
扩写润色	`/api/polish`	polish 任务	单次 LLM
知识库（RAG）	`/api/knowledge`	RAGSearchTool（被审查 Agent 调用）	独立检索服务
经验自学习	审查完成后异步	experience/ 模块	数据管线（Case→Skill）
开放 API（第三方 AI 助手）	`/api/v1/open`（X-Api-Key）	复用审查/查重	见 `skill/bjt-bid-review/`

所有消耗 LLM 的能力都走**统一计费链路**（点数制），这是本系统区别于演示项目的工程重点，见 §7.4。

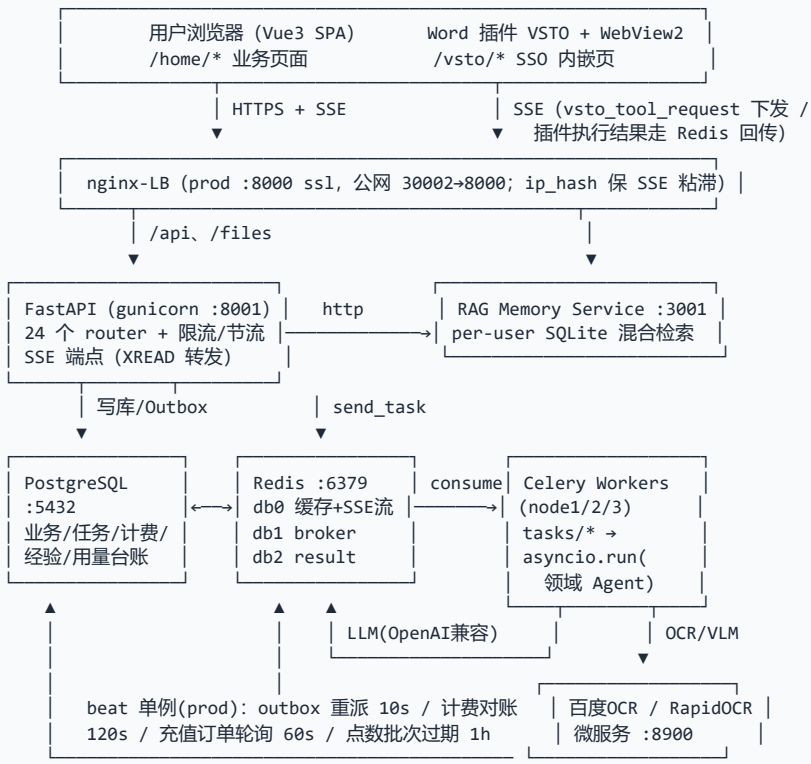
1.3 技术栈

层	技术
后端 API	Python / FastAPI / SQLAlchemy(async) / slowapi / pydantic BaseSettings
任务队列	Celery（broker=Redis，beat 定时，三队列隔离）
Agent 框架	Mini-Agent 子模块（MiniMax 开源，ReAct 循环 + Tool/LLMClient 抽象）
LLM	OpenAI 兼容协议；provider 可配 minimax / volcengine / deepseek / tencent，生产现为腾讯 TokenHub（deepseek 系模型）
主库	PostgreSQL 15（docker），手写编号 SQL 迁移
缓存/队列/事件	Redis 7（docker），分 db0/1/2 三个用途
向量检索	rag_memory_service（Node/Express + better-sqlite3 + sqlite-vec + FTS5，智谱 embedding-3），**不用 pgvector**

层	技术
文档解析	docling / markitdown / mammoth / MinerU（third_party 子模块）；OCR：百度云 + 本地 RapidOCR 微服务(:8900)
前端	Vue 3.4 + TypeScript 5 + Vite 5 + Pinia 2，Element Plus + Ant Design Vue，marked/mermaid 渲染报告
部署	API/Celery 原生进程（gunicorn + systemd），PG/Redis docker；4 机集群

---

2. 总体架构



分层职责一句话版：

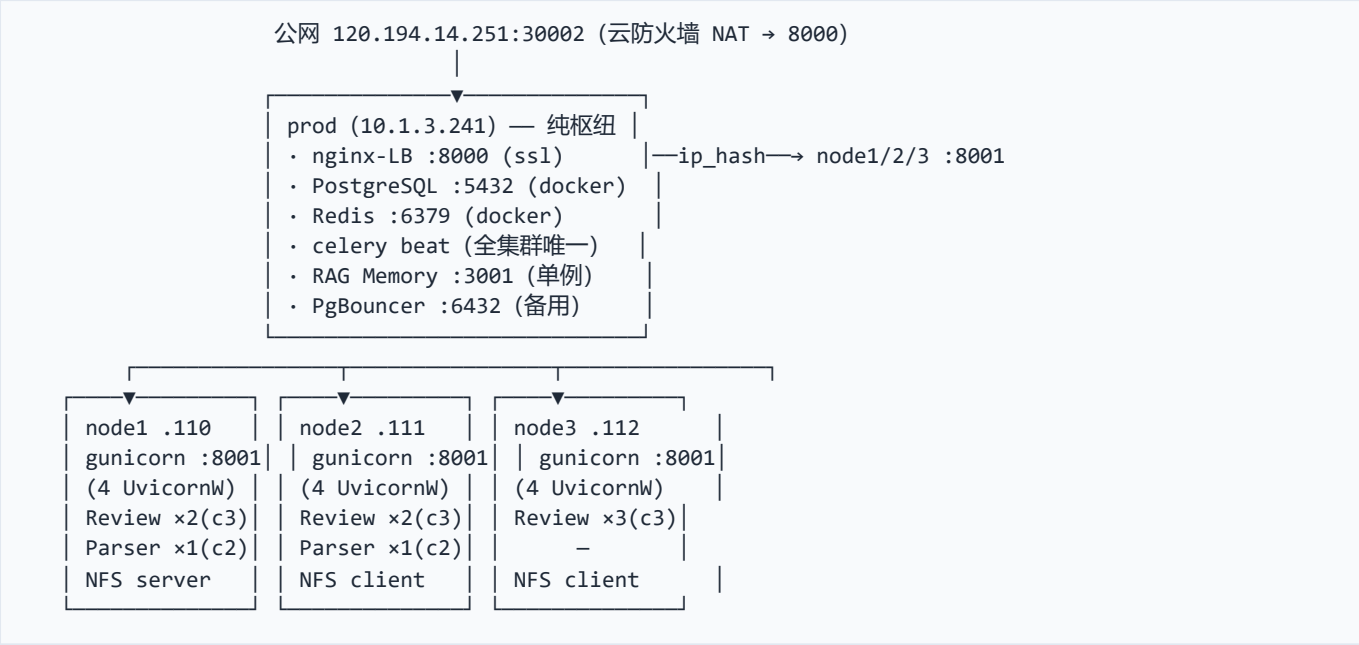
- **API 层**（`backend/api/`）：鉴权（JWT + 运营台外包登录）、参数校验、**计费授权闸门**、建任务行 + Outbox、立即返回 task_id。不做任何重计算。
- **任务层**（`backend/tasks/`）：Celery 任务是所有重计算的入口，负责认领、超时控制、心跳、事件流、结果落库、计费终局。每个任务体内 `asyncio.run()` 进入异步世界。
- **Agent 层**（`backend/agent/`）：领域智能体与编排器，消费 Mini-Agent 框架的循环能力。
- **工具层**（`backend/agent/tools/` + `backend/services/`）：Agent 的手和脚——文档检索、比对、RAG、OCR、VSTO 远程 Word 操作、用量计量。
- **数据层**：PG 是唯一事实源（业务状态 + 计费台账）；Redis 承担队列/事件/缓存/取消标记四职；RAG 服务独立持有向量索引。

关键设计立场：**LLM 只做判断，不做记账。**所有金额、点数、状态流转由确定性代码完成，LLM usage 只是记账的原始凭证（§7）。

---

3. 生产部署拓扑

权威源：集群部署方案.md §3.9。要点如下（IP 只写内网，无凭证）：



- **prod 不跑计算**：Phase 4 后 prod 只保留边缘与 DB/单例服务。计算全在 node1/2/3。
- **workspace 文件共享**：NFS 实体在 node1 `/srv/bjt_workspace`，软链到各节点 `/home/bjt/bjt_agent/workspace`；**prod 不挂载 NFS**——这是"prod 禁止起 worker"的根因（prod 假 worker 抢到任务后读不到文件，全部误判失败，08-28 事故）。
- **队列容量**：Review 槽位 (2+2+3)×3 = 21（目标并发 20）；Parser 2×2 = 4。
- **worker 保护参数**（启动脚本 `scripts/bjt-cluster.sh`）：`--max-tasks-per-child=5`、Review `--max-memory-per-child=2500000`(约 2.5G)、Parser 2G——用进程自杀换内存泄漏免疫。
- **Redis 分库**：db0 = 业务缓存 + SSE Stream（`sse:stream:<task_id>`）；db1 = celery broker；db2 = celery result。
- **nginx SSE 要点**：upstream `ip_hash` 做会话粘滞（SSE 长连接必须落在同一节点）、`proxy_buffering off`、`max_fails=3 fail_timeout=30s` 自动摘除故障节点。
- **节点启动脚本**：`scripts/bjt-cluster.sh`（节点）/ `scripts/bjt-proc.sh`（prod 单例）。集群维护铁律见集群维护速查手册.md（如 rsync 必须排除 `.env`、SSH 远程操作一律脚本经 stdin 注入等）。

单机形态（`docker-compose.yml`）只有 PG15 + Redis7 两个容器，backend/celery 仍是原生进程——**API 与 Worker 不容器化**是全形态共识。

---

4. 核心框架层：Mini-Agent

4.1 定位与引入方式

`Mini-Agent/` 是 MiniMax-AI 开源的单 Agent 演示框架（MIT）的 git 子模块，提供"极简但专业"的 ReAct 循环骨架。引入方式不是 pip 安装，而是 **sys.path 注入**：

`backend/utils/mini_agent_utils.py::setup_mini_agent_path()` 把仓库根的 `Mini-Agent/` 插入 `sys.path`，随后 `import mini_agent`。所有需要它的模块（`bid_review_agent.py`、`llm_factory.py` 等）在 import 前先调它。

> 工程提示：本机克隆后必须 `git submodule update --init Mini-Agent`，否则 `No module named mini_agent`（`.gitignore` 相关坑，见 sales 工作树 AGENTS.md）。

4.2 四个核心抽象

抽象	位置	要点
`Agent`	`Mini-Agent/mini_agent/agent.py`	单 Agent 执行循环。构造参数 `llm_client, system_prompt, tools, max_steps=50, workspace_dir, token_limit=80000, event_callback`
`Tool`	`Mini-Agent/mini_agent/tools/base.py`	抽象属性 `name / description / parameters`（JSON Schema）+ `async execute() -> ToolResult(success, content, error)`；自带 `to_schema()`（Anthropic）/ `to_openai_schema()`（OpenAI function calling 双格式）
`LLMClient`	`Mini-Agent/mini_agent/llm/llm_wrapper.py`	anthropic/openai 双 provider 门面；支持 `reasoning_split`（思维链分离进 `Message.thinking`）、`reasoning_mode="deepseek"`、内建指数退避重试
`Message` / `LLMResponse`	`Mini-Agent/mini_agent/schema/schema.py`	`Message(role, content, thinking, tool_calls, tool_call_id)`；`TokenUsage` 含 DeepSeek 风格缓存命中/未命中拆分

工具注册机制非常朴素：构造时 `self.tools = {tool.name: tool for tool in tools}`——一个按 `Tool.name` 字符串索引的 dict。后续可随时 `self.tools[name] = tool` 动态补注册（backend 的 MCP 工具、视觉工具都这样注入）。

4.3 ReAct 执行循环（`Agent.run()`）

每个 step 的固定节奏：

```
while step < max_steps:
    1. 发 step_start 事件 → 检查 cancel_event（取消安全点①）
    2. await _summarize_messages()          # 上下文记忆化（见 4.4）
    3. response = await llm.generate(messages, tools)
    4. append assistant Message（含 thinking / tool_calls）
    5. 无 tool_calls → 任务完成，返回最终文本
    6. 逐个执行工具：
       - 未知工具   → ToolResult(success=False, error="Unknown tool")  # 不抛异常
       - 执行异常   → 捕获全部异常 → 失败 ToolResult（带 traceback）     # 不中断循环
       - append role="tool" 消息 → 检查取消（安全点②）
    7. 发 step_complete 事件
耗尽 max_steps → 返回 "Task couldn't be completed after N steps."
```

对 Agent 工程师最重要的三个特性都藏在这个循环里：

1. **异常即工具结果**：工具层的任何崩溃都变成一条失败消息回给 LLM，让它自己决定重试还是换路——循环永不因工具异常而中断。
2. **事件协议**：`step_start / llm_output / tool_call_start / tool_call_end / step_complete / completed / cancelled / max_steps_reached`，经 `event_callback` 抛出，是 SSE 实时进度的唯一来源（领域 Agent 覆写 `_emit_event` 做聚合，见 §5.1）。
3. **取消安全点**：只在 step 边界与工具前后检查 `cancel_event`，即"协作式取消"，保证消息列表一致性（`_cleanup_incomplete_messages()` 清理半截消息）。

4.4 上下文记忆化（`_summarize_messages`）

- token 计量：tiktoken cl100k_base 精确计数，失败退化为 `chars/2.5` 估算；本地估计与 API 上报 `total_tokens` **任一**超 `token_limit` 即触发。
- 压缩策略：**保留 system + 全部 user 消息**，把相邻 user 消息之间的执行过程（assistant + tool 消息）用一次额外 LLM 调用压缩成 `[Assistant Execution Summary]`。
- 审查 Agent 的 `token_limit` 配到 800K（`config.py: agent_token_limit`），配合 max_steps=500 支撑长文档审查。

4.5 框架与领域的边界（三个接缝）

backend 不改框架源码，通过三个接缝组合出领域能力：

接缝	用法	例子
继承 Agent	领域 Agent 继承 `mini_agent.agent.Agent`，注入领域提示词与工具集，覆写 `_emit_event`、包裹 `llm_client.generate`	`BidReviewAgent`、`BlindCheckAgent`
扩展 Tool	领域工具继承 `mini_agent.tools.base.Tool`，但用 backend 自己的 `ToolResult`（多一个 `data: dict` 字段供代码消费结构化结果）	`backend/agent/tools/*` 全部
复用 LLMClient/Message	裸 LLM 调用（无工具循环）也走同一客户端与消息 schema，保证计量/重试/协议统一	`llm_factory.create_llm_client()`、experience 模块

一句话分工：**Mini-Agent 管"循环、协议、抽象、摘要、事件、取消"；backend 管"领域提示词、领域工具、编排、持久化、计费、SSE 落地"**。单 Agent 时代的设计文档见 mini-agent-interaction.md（描述的是早期单文档形态，现已演进为多文档 + 结构化工具 + max_steps=500）。

---

5. 领域 Agent 层：六种形态

`backend/agent/` 下所有叫 "Agent" 的东西**并不是同一种东西**。这是本系统最重要的架构判断：**按任务性质选择 Agent 形态，而不是把一切塞进 ReAct 循环。**

Agent	继承 Mini-Agent?	形态	适用判断
BidReviewAgent	✅	完整 ReAct 工具循环	开放式检索 + 判断，路径不可预知
BlindCheckAgent	✅	固定编排 + 受限 ReAct + 守门员重问	检查项确定性高，但需 LLM 组织结论
BidDraftAgent（V1 生成）	❌	纯代码编排的多阶段 LLM 管线	输出结构固定，无需自主探索
bid_wizard 系列（V2 编标）	❌	素材索引→问卷→Spec→逐章，代码编排	同上，且阶段间需人工介入
DuplicateCheckAgent	❌	确定性检索 + 单次结构化 LLM 判定	证据必须可复现，LLM 只做裁决
MasterAgent / DuplicateMasterAgent	❌	纯 Python 编排器（不调 LLM）	扇出并行与聚合

5.1 BidReviewAgent —— 完整 ReAct 审查子代理

`backend/agent/bid_review_agent.py`（约 2400 行，系统核心）。它审查**一个规则文档**（一个 TodoItem），由 MasterAgent 并行调度多个实例。

构造与工具集：`max_steps=500`；工具 = `search_tender_doc`（关键词行搜索，8000 字符分块、±5 行上下文、图片引用邻近扫描、文件级缓存）+ `rag_search` + `compare_bid`（工具内嵌独立 LLM 做单条合规判定）+ `decide_merge` + `get_document_toc / get_section_content / get_section_images / get_image_ocr`（结构化工具共享 loader，避免重复读文件）+ `understand_image`（按 provider 换实现：百度 OCR / 火山视觉 / MCP）+ `write_file / read_file`（复用框架 file_tools）。

LLM 调用包裹（`_wrap_llm_generate`，对 `llm_client.generate` 的 monkeypatch）——每次 LLM 调用依次经过：

1. Redis 分布式滑窗限流（集群 LLM 并发上限）；
2. 进程内信号量（`max_llm_concurrency`，跨事件循环重建）；
3. `asyncio.timeout(180s)` 单次超时；
4. TimeoutError 专项重试（默认 2 次）；
5. 成功后：更新任务心跳 → 检查取消 → 写 Redis 指标 → **`record_llm_usage` 计费记账** → 交互日志（最终落 `logs/interaction_*.jsonl` 供复盘）。

运行时自治机制：

- **心跳**：每次 LLM 调用/step 后更新 `ReviewTask.last_heartbeat`；后台协程 5s 轮询，API 取消标记优先，超时即 set `cancel_event`。
- **防重复动作**：`deque(maxlen=20)` 记录 (tool, key_arg) 签名，同签名 ≥3 次即注入中文 `[系统提醒]` user 消息——用 prompt 干预替代硬熔断。
- **事件聚合**：覆写 `_emit_event`，把细粒度事件聚合成一条 `sub_agent_step`（含 step_type/content/tool_calls/tool_results）供前端时间线。

结果抽取的兜底链（`_post_process`）：① 规则解析 Agent 写出的 markdown 报告（固定小节结构 `## 检查项 N`）→ ② 失败则 LLM 从消息历史抽取 findings → ③ 再失败则关键词扫描（含合规否定短语防误报）→

severity 由规则推断。空 findings 是**显式语义**（全部合规），有专门的抑制逻辑防止关键词兜底把"全部合规"误报成违规。

5.2 MasterAgent / SubAgentExecutor —— 编排器模式

`backend/agent/master/master_agent.py`。**纯 Python 编排器，自己不调 LLM**。四阶段：

```
1. RuleLibraryScannerTool 扫描规则库 *.md（含 GBK 文件名自愈）
2. 用户勾选大类过滤 → 每个规则文档建一个 TodoItem（session 内可观测）
3. asyncio.Semaphore(max_sub_agent_concurrency) + gather(return_exceptions=True)
   并行执行 SubAgentExecutor.execute（每 Todo 一个 BidReviewAgent）
4. 纯统计聚合（不做 LLM 合并）→ report_agent 生成总体报告
```

`SubAgentExecutor`（`master/sub_agent_executor.py`）是子代理的"保姆"：

- **用量归属**：入口 `set_usage_context(UsageContext(task_id, todo_id, user...))`（contextvars，天然隔离到本协程树），出口 finally `reset_usage_context`——计费台账由此能精确到"哪个规则文档花了多少钱"（§7.3）。
- **重试语义**：`max_steps` 耗尽 **fail-fast 不重试**（避免雪崩）；`detect_anomaly`（write_file 没被调用且 findings 空 = 异常行为）触发指数退避重试 2/4/8s；"被调用且空"判为全部合规不重试。
- **落库**：`sub_agent_step` 事件异步写 `agent_steps` 表（时间线）；每个子代理完成即增量写 `ReviewResult`（前端无需等全部完成）。

为什么 Master 不用 LLM 编排？ 规则库是结构化目录，扇出关系确定，用代码编排可获得：确定性（同样输入同样扇出）、免费（不耗 token）、可观测（TodoItem 即进度条）。LLM 编排只在"计划本身需要推理"时才值得。

5.3 BlindCheckAgent —— 确定性优先的暗标检查

`backend/agent/blind_check_agent.py`。暗标检查 = 验证投标文件格式是否满足招标文件的无标识要求（字体、字号、行距、页边距、标题层级...）。这个领域吃过两次大亏（2026-09-12 大面积假阳性、09-17 大文档超时），现行设计是 ADR-0002/0003（位于 sales 工作树 `docs/adr/`）的落地，**教科书级的"确定性优先 + LLM 收口"**：

1. **工具参数从要求文本解析，绝不落默认值**（`_build_deterministic_arguments`）：纯正则解析字体名（含 `仿宋_GB2312` 归一）、字号（中文字号名→磅值表）、RGB/黑字、行距（固定值/倍数）、页边距、标题层级上限→锚定正则。**解析不出的维度显式跳过并产出"待确认"note**——历史事故根因就是工具参数硬编码样例值（28磅/2.5cm）。
2. **回显校验**（`_apply_echo_verification`）：插件回传的 `expected` 与传入意图逐维度比对，错配维度的"违规"被剔除——防旧版插件用内置默认值产生假阳性。
3. **强制证据先行**：先 `word_get_overview` 拿 snapshot_id，再固定采集身份线索（6 类），然后才进 ReAct。
4. **时间预算分阶段**：总预算 25min，证据采集最多占 60%，LLM 收尾预留 150s；全文扫描超时按文档规模自适应（1500 字符/s + 0.01s/段，clamp 90–480s）。
5. **超预算降级**：ReAct 超时 → `_final_summary_without_tools`（不再调工具的一次性汇总）；模型漏掉的违规维度由"守门员重问"（`_guardrail_reask`）带事实追问一次，仍未覆盖的降级为"待确认"卡——**"AI 是用**

户可见发现的唯一作者"（ADR-0002：绝不把工具原始违规直接展示给用户）。

6. **结果口径**：只分「严重」「待确认」两类；无违规但覆盖不完整 → overall=unknown（不假装合规）。

5.4 BidDraftAgent / bid_wizard —— 代码编排的生成管线

V1 BidDraftAgent（`bid_draft_agent.py`）：文件头注释明言不走 Mini-Agent 循环。四阶段：`_analyze`（招标要素提取 JSON）→ `_build_outline`（大纲规范化：层级编号、剥重复前缀、clamp 段落数，上限 60 节）→ `_generate_section` 逐节撰写（产物做 markdown 围栏修复、每节 mermaid 上限 2 张）→ `_assemble` 拼装。写作红线内置于 prompt：**严禁虚构，事实空白写（请补充）**。

V2 AI 编标（`bid_wizard_agent.py`，复用 V1 内核 import 而非复制）新增三件武器：

- **素材两阶段注入**：确定性分块（标题感知，1500 字符/块，上限 200 块）→ LLM 为每块写摘要/关键词 → `index.md + chunks/NNN.md`。写作时**再由一次 LLM 按章节挑选分段号**，代码读选中分段入上下文——避免把全部素材塞进 prompt。
- **多轮问卷**：AI 生成 6–15 题（建议答案优先来自素材事实），自判是否追问；用户答案四态（answered/adopted/skipped/supplemented）合并。
- **Spec 硬约束**：商务/技术部分开关、目标总字数 ±20%、配图开关；平级劣化检测 + 带纠正指令重试一次。
- **JSON 容错链**（`WizardLLM.generate_json`）：宽容解析（`raw_decode` 取首个完整 JSON）→ 时间预算允许则带纠正语重试 → `salvage_truncated_json`（括号栈扫描，保留已完整元素救援截断输出）→ 才报错。这是对抗长 JSON 输出被 max_tokens 截断的实用模式。

5.5 DuplicateCheckAgent —— 确定性检索 + 单次裁决

查重的证据必须可复现、可法庭级解释，所以 LLM 被压到最小面：

```
search_duplicate_candidates（确定性检索，S2 多通道：哈希/词法/结构/图像/语义）
  → 规则过滤 → top-N 补上下文 → 来源候选检索
  → 单次结构化 LLM 判定（JSON 校验失败带纠正语重试 1 次）
  → _materialize_findings：模型决策绑回不可变候选证据
      （相似度/原文/位置不得改—LLM 只能判定，不能改证据）
  → 正则拦截"串标/围标"等法律结论（输出词表管控）
```

外层 `DuplicateMasterAgent` 同 Master 模式：规则（D001–D010）→ Todo → 信号量并行 → finding 与 Todo 状态同事务落库。

5.6 QualityEvaluator / report_agent —— LLM-as-Judge 与确定性报告

- **QualityEvaluator**（`quality_evaluation.py`）：三条维度各一次 LLM 调用——合规准确性 40% / 严重程度恰当性 30% / 完整性 30%，加权出 finding 级分数，批次均值 ≥70 通过。它同时服务于质量门禁与经验系统的 `quality_score`（§11）。
- **report_agent**：**统计、分组、废标风险评级全部由代码确定性计算**，LLM 只做三件事（风险描述合并、废标语义判定、评分类得分提取），且 LLM 输出逐项校验、缺失就地降级为原文摘录拼接（`degraded=true`）。

---

6. 工具层设计

6.1 工具抽象

backend 工具继承 `mini_agent.tools.base.Tool`（`name/description/parameters` + `async execute()`），但返回 backend 扩展的 `ToolResult`（多 `data: dict` 字段）——**content 给 LLM 读（字符串），data 给代码消费（结构化）**，两者职责分离。所有工具遵循**软失败**约定：404/超时/空结果都返回带建议的 `success` 结果，绝不让 Agent 循环因工具故障而中断（`rag_search.py` 是范本）。

6.2 工具清单（`backend/agent/tools/`）

工具名	文件	作用
`search_tender_doc`	doc_search.py	关键词行搜索（中文参数名兼容），分块 + 上下文 ±5 行 + 图片引用提取 + 缓存
`rag_search`	rag_search.py	HTTP 调 RAG 服务 `/api/search`（X-User-ID 头），软失败
`compare_bid`	comparator.py	**工具内嵌独立 LLM** 做单条合规判定（JSON 多策略解析）
`decide_merge`	merge_decider.py	LLM 合并决策 keep/replace/discard
`get_document_toc` / `get_section_content` / `get_section_images` / `get_image_ocr`	structure_tools.py	markdown 标题树切章节，按节取内容/图片/OCR
`understand_image`	baidu_ocr.py / volcengine_vision.py	同名工具按 provider 占位覆盖（百度版各出口记 OCR 用量）
`search_duplicate_candidates` 等 4 个	duplicate_candidates.py	查重候选检索/上下文（薄封装 service 层）
`word_*` ×12	vsto_remote.py / vsto_chunked.py	VSTO 远程 Word 工具（见 6.3）
`write_file` / `read_file`	Mini-Agent file_tools	审查报告产物落 workspace

另有 MCP 集成：`backend/mcp.json` 声明 `minimax_coding` MCP server（stdio，提供 understand_image/web_search），仅在 provider=minimax 默认路径经 `load_mcp_tools_async` 加载（在 Celery worker 里起 MCP 子进程需要 LoggingProxy/anyio 兼容补丁，见 `bid_review_agent.initialize()`——这是把桌面框架搬进 worker 的代价样本）。

6.3 VSTO 远程工具协议（本系统最独特的部分）

问题：暗标检查需要读取用户**本机 Word 进程里**的实时文档对象（字体、行距、分节……），而 Agent 跑在云端 Celery worker 里。

架构：云端 Agent ↔ PG + Redis + SSE ↔ 用户 Word 内的 VSTO 插件（WebView2 桥）。Agent 调一个 `word_*` 工具 = 向用户 Word 发一次 RPC。

调用时序（`backend/services/vsto_tool_broker.py::VstoToolBroker.request`）：



这套协议里沉淀的防呆设计，每一条对应一次真实事故，值得逐条学习：

机制	防的是什么
工具 schema 白名单闭式注册（"adding a tool requires a protocol review"）	插件与云端能力集漂移
快照绑定 + 回包快照一致性校验	用户在检查中途切换文档，结果张冠李戴
requested_at + expires_at 成对下发，插件用本机单调时钟判过期	用户机时钟偏差几分钟导致整体误判（墙钟不可信）
进度心跳续期（45s 空闲窗口 + 600s 绝对上限）	长扫描被固定超时误杀（ADR-0003：466 页文档 21 次调用仅 5 次成功的事故）
ChunkedVstoTool 分块窗口：首窗口探测耗时，后续按 ~25s 目标自适应窗口大小；失败窗口减半重试一次，仍失败记 uncovered gap；违规上限 300 条	单次全文扫描超时 + Word 串行队列雪崩
旧插件兼容：不认范围参数时按 `scope` 回显识别并直接采用全文结果	新旧插件并存期的协议平滑
迟到结果找回（`fetch_late_result`）	超时判定后结果才到，白扫一遍
memo 缓存 + 熔断器	同参重复调用打爆用户 Word；插件已死还在发

---

7. LLM 调用与用量治理

7.1 Provider 工厂

`backend/services/llm_factory.py::create_llm_client()` 按 `settings.llm_provider` 构造 Mini-Agent `LLMClient`，全部走 **OpenAI 兼容协议**（volcengine / deepseek / tencent / minimax；deepseek 与 tencent 加 `reasoning_mode="deepseek"` 解析思维链）。当前生产：`LLM_PROVIDER=tencent`（腾讯 TokenHub，`tokenhub.tencentmaas.com/v1`，deepseek 系模型）。切换 provider = 纯 `.env` 操作 + 滚动重启，但**必须先验 usage 字段风格**（见 §12 教训）。

7.2 重试 / 限流 / 超时（三层防御）

```
① Mini-Agent 内层 async_retry      指数退避 1s 起、上限 60s、默认 3 次（网络/5xx/429）
② BidReviewAgent 包装层             TimeoutError 专项重试（180s 单调超时 × 2 次重试）
③ 编排层业务重试                    各编排器自带 2-3 次带纠正语重试（JSON 类输出）
限流：Redis 分布式滑窗（集群级 LLM 并发上限）+ 进程内信号量（单 worker 级）
```

7.3 用量计量管线（从一次 generate 到一条台账）

```
SubAgentExecutor.execute 入口
  └ set_usage_context(UsageContext)        # contextvars: external_user/local_user/
                                           #   project/task/todo + 企业维度（反查 User）
      └ Agent 每次 llm.generate（被包装）
          └ usage_recorder.record_llm_usage
              ├ usage: TokenUsage（含缓存命中拆分）
              ├ cost_calculator.estimate_cost → cost_cny   # 厂商标价表唯一来源；
              │                                            #   只对 success 计费
              └ 异步写 ai_usage_records（NullPool 短连接 × 3 次退避重试）
                  └ fire-and-forget refresh_task_summary   # 增量聚合到
                                                           #   ai_usage_task_summary
```

两个关键工程细节：

- **NullPool 双引擎**（`backend/models/base.py`）：API 路径用池化 engine；Celery 路径的用量写入用 **NullPool 短连接**。原因：Celery 任务内 `asyncio.run()` 每次新建事件循环，池化连接绑定死循环会报 "Event loop is closed"。
- **计量接缝有两种**：审查 Agent 用自研 `_wrap_llm_generate`（顺带做限流/心跳/指标）；其余 Agent（暗标/编标/生成/润色）用通用 `instrument_llm_client()` 包装——同一记账出口，两种插桩方式。

7.4 从计量到扣点：可计费任务三段式

所有花钱的任务（审查/查重/暗标/生成/编标/润色/开放 API）走同一生命周期（`backend/services/task_lifecycle.py`）：

```
【第一段·授权】API 层 authorize_billable_task_start
   ├ ensure_wallet(FOR UPDATE)          # 钱包行锁 = 账户级串行化点
   ├ 余额 ≤ 0 → 402；未结算任务 ≥ 3 → 409  # 结算未完成不放行新任务
   └ 同一事务写 任务行(billing_status=pending) + TaskDispatchOutbox
【第二段·执行】worker claim_task_for_execution(FOR UPDATE 认领，防重复消费)
   └ 跑 Agent … 每笔 LLM 调用实时记账
   └ 终态 finalize_task_usage:
       flush_task_usage（等全部流水落库，失败→billing_status=retry）
       → refresh_task_summary(strict) → enqueue settle_task_billing
【第三段·结算】billing_tasks.settle_task_billing（max_retries=8 指数退避）
   └ billing.settle_task_consumption:
       幂等（已有 ConsumptionRecord 直接 settled）
       SUM(ai_usage_records.cost_cny WHERE task_id) × billing_multiplier → 点数
       allocate_consumption: recharge 批次先扣、gift 后扣（逐 lot 分配）
       写 ConsumptionRecord + WalletTransaction → billing_status=settled
兜底：beat 每 120s reconcile_task_billing 回收孤儿终态任务
```

设计立场：**失败/取消的任务照样结算**（成本已真实发生，`billing.py` 有意为之）；**绝不产生 0 成本消费记录**（flush 失败保持 retry 状态等重试，而不是按 0 元结算）。Redis 全挂也不影响已落库流水的结算正确性——outbox + 对账保证最终送达。

---

8. 任务工程与可靠性

8.1 Celery 拓扑

`backend/celery_app.py`：app 名 `bid_review_agent`，broker/result 均为 Redis。

队列	消费内容	隔离理由
`review`	审查/查重/暗标/润色/计费/反馈/经验	主业务
`parser`	仅 `parse_document`	解析吃 CPU/内存，不与审查抢 worker
`generation`	bid_draft / bid_wizard	长生成任务（小时级）不阻塞审查

> **铁律：新增 Celery 任务必须同步补 `task_routes`**。漏配则 `.delay()` 直发默认 `celery` 队列，专用 worker 不消费，**消息滞留且无任何报错**（排查：`redis-cli LLEN celery` 非零即中招）。

beat 定时（全集群单例，跑在 prod）：outbox 重派 10s / 计费对账 120s / 充值订单轮询 60s / 充值单过期 300s / 点数批次过期 3600s。

worker 级配置：`prefetch_multiplier=1`（防饥饿积压）、`task_time_limit=28800` 全局兜底、`task_annotations` 按任务分级限时（review 5700/6000s、bid_draft 6900/7200s）、broker 无限重试 + keepalive + `visibility_timeout=3600`（对抗第三方 Redis 频繁断连）。每个任务体用 `asyncio.run()` 进入异步内核。

8.2 Outbox 模式（任务可靠派发）

"建任务行 + 发 MQ"若分两步，就有"行落库但消息没发出去"的窗口。本系统的解法：

```
同一 DB 事务：INSERT 任务行 + INSERT TaskDispatchOutbox
提交后 best-effort：celery send_task + 标记 dispatched
发送失败：outbox 行留 pending/retry → beat 每 10s dispatch_pending_task_outbox 重派（指数退避）
```

配合 worker 侧 `claim_task_for_execution`（`SELECT ... FOR UPDATE` 认领、状态机校验），实现**至少一次投递 + 恰好一次执行**。

8.3 心跳、取消、超时（一组协作机制）

机制	实现	说明
DB 心跳	Agent 每次迭代更新 `ReviewTask.last_heartbeat`；`agent_progress_timeout=1200s`	进度活性判定；前端断连/关页超时自动取消
取消标记	Redis `set_task_cancelled / is_task_cancelled`	API 取消入口写；**SIGTERM 优雅停机也写它**（worker 收到 kill 先转成业务取消而非僵死）
取消传播	Mini-Agent `cancel_event`（协作式，§4.3）	逐层 set，Agent 在安全点自停
绝对兜底	任务级 `asyncio.wait_for(agent_total_timeout=21600s)` + Celery time_limit	任何单点失灵也有上限

机制	实现	说明
僵死清理	启动时 pending/running 超 45min 标 failed + `/api/tasks/cleanup-on-restart`（HMAC 内部端点）	重启后收尸 + 计费回退重派对账

8.5 SSE 实时进度

```
Agent 事件回调 → _publish_event → Redis Stream sse:stream:{task_id}
                                  (XADD + Lua 设 TTL 1h + MAXLEN 1000)
浏览器 GET /api/events/tasks/{id}/stream
  → 鉴权：EventSource 不能带 Authorization 头 → query token + 任务归属校验
  → 每连接独立 async Redis 客户端 XREAD + 应用层心跳
  → 外部用户过滤 sub_agent_* 明细事件（只给进度计数）
nginx：ip_hash 粘滞 + proxy_buffering off（否则 SSE 被缓冲）
step 事件同时落 agent_steps 表（时间线持久化，刷新页面可回放）
```

---

9. 数据架构

9.1 PostgreSQL（唯一事实源，约 40 张表）

按表域分组：

表域	表	说明
用户/项目	users、projects、api_keys	登录外包运营台，本地 find-or-create
文档	documents	doc_type=tender/bid；parsed_markdown_path / parsed_images_dir 是 Agent 的食物
审查	review_tasks、review_results、agent_steps、review_share_tokens、project_review_results	心跳列、时间线、分享
查重	duplicate_results / occurrences / pair_summaries / evidence_clusters / document_members	证据不可变原则
暗标	blind_check_tasks、blind_check_findings	只分严重/待确认
生成/编标/润色	bid_draft_tasks/sections、bid_wizards/materials/index_tasks/qa_tasks/writing_tasks、polish_tasks	向导状态机
计费可靠性三件套	**task_dispatch_outbox**（派发）、**ai_usage_records / ai_usage_task_summary**（用量台账）、**consumption_records / wallet_transactions / billing_orders / user_wallets**（钱包流水）	§7.4 全链路的物证
销售	sales_configs / sales_packages / grant_batches / credit_lots / point_ledger_entries / consumption_allocations	点数批次（recharge/gift）与分配
经验	experience_cases / skills / cluster_membership / feedback	§11

表域	表	说明
运维	system_announcements、system_maintenance、vsto_tool_sessions / tool_calls	VSTO 调用单据

并发控制一律 `SELECT ... FOR UPDATE`（钱包行锁、任务认领锁、结算锁）。连接层注意：API 与 Celery 双引擎（§7.3）；PgBouncer 预留未启用。

迁移机制：47 个**手写编号 SQL**（`backend/migrations/NNN_描述.sql`），按运维手册 `docker exec bjt-postgres psql` 手工执行；alembic 仅在依赖里声明、未启用；新环境靠启动时 `create_all` 兜底建表。由此产生的新表铁律见 §13.4。

9.2 Redis 的四个角色

① Celery broker + result（db1/db2）；② SSE 事件总线（Stream，db0）；③ 限流计数（slowapi 存储，强制 RESP2 兼容老 Redis）；④ 任务取消标记 + LLM 滑窗限流 + VSTO 工具结果信箱（`getdel` 单次消费）。**Redis 定位为"可丢失的加速层"**：挂了影响实时性，不影响正确性。

9.3 RAG 知识库服务（独立服务，与 PG 无关）

```
rag-memory/ (vendored TS 库: better-sqlite3 + sqlite-vec 向量 + FTS5 全文)
rag_memory_service/ (Express HTTP 封装, :3001, 生产单例跑在 prod)
  · 多租户: 按 X-User-ID 懒建索引, {DOCUMENTS_PATH}/{userId}/data/memory.sqlite
  · 混合检索: vector 0.7 + BM25 0.3; embedding=智谱 embedding-3 (1024 维)
  · 路由: POST /api/search (必带 X-User-ID) / POST /api/sync (DOCX/PDF 先转 MD 再索引)
          / GET /api/status / GET /api/readfile
backend 侧消费: RAGSearchTool (Agent 工具, 软失败) + knowledge API + 经验检索召回
```

选型立场：**不引入 pgvector**，知识库是"每用户私有文档集"，per-user SQLite 文件天然隔离、可整体备份/删除，与主库业务数据完全解耦。

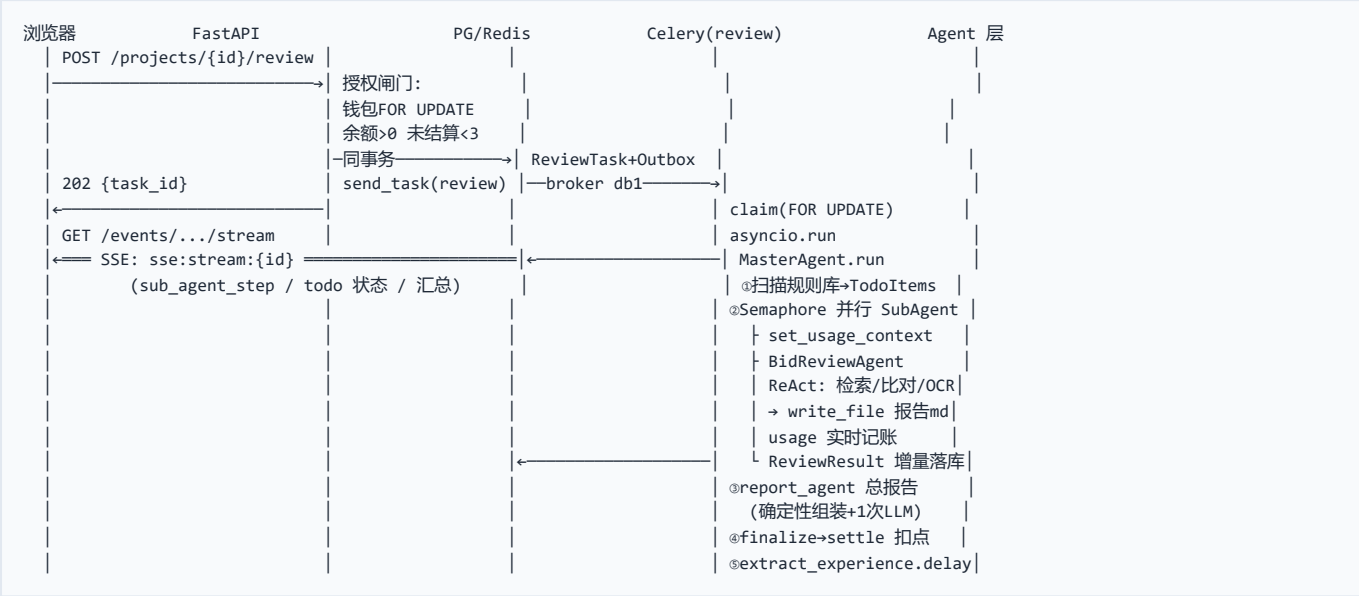
9.4 OCR 与文档解析

- **ocr_service/**：独立 FastAPI 微服务（RapidOCR，:8900，`POST /api/ocr` base64→文本），本地兜底通道；生产图像理解主用百度云 OCR（按次计量计费）。
- **third_party/** 子模块：docling / markitdown / mammoth / MinerU / pdf.js。`backend/parsers/` 按类型分派：PDF→docling（表格结构+图片提取）、docx→mammoth、xlsx、图片→OCR/VLM。每任务新建独立 engine（`document_parser.py`）。
- 解析结果落 workspace：`parsed.md + _images/` ——**markdown 是全系统统一的文档中间表示**，Agent 的所有检索/结构化工具都吃它。

---

10. 核心工作流程走读

10.1 标书审查（主链路）



取消：前端断开心跳或点取消 → Redis 标记 → Agent 心跳协程 set cancel_event → 协作式停止 → 终态 cancelled（照常结算）。

10.2 暗标符合性检查

上传投标文件（普通上传，不走计费闸门）→ 用户在插件/网页发起暗标检查（计费三段式）→ worker `run_blind_check`：建 VSTO 会话（绑定文档快照）→ `word_get_overview` → 确定性参数解析 + 固定身份线索采集 → 受限 ReAct（预算=总预算−150s，工具经 ChunkedVstoTool 分块扫 Word）→ 回显校验 → 守门员重问 → 归一化（严重/待确认）→ findings 落库 + SSE。全程插件经 SSE 收 `vsto_tool_request`、执行结果写 Redis。

10.3 AI 编标（wizard）

上传素材 →（不走计费闸门）`run_bid_wizard_index`：分块→逐块摘要→index.md（generation 队列）→ 多轮问卷（同步 API，人机交互）→ `generate_spec`（硬约束+劣化检测重试）→ 用户确认 Spec → `run_bid_wizard_write`（generation 队列）：逐章生成，上下文=素材摘录+前 12 章标题摘要（支持断点续作）→ 逐章落库，前端拼装导出。

10.4 技术标查重

项目内上传多份技术标 → `run_duplicate_check`（review 队列）→ 文档解析 → 逐文档建 SourceIndex → 按规则（D001–D010）扇出 DuplicateCheckAgent（Master 模式）→ 确定性候选检索 → 单次 LLM 判定 → 证据物化落库（duplicate_occurrences / evidence_clusters）→ 报告。

10.5 文档解析（一切的前置）

上传（upload_throttle 64KB 分块限速 + 1GiB 上限）→ `parse_document.delay`（parser 队列，不计费闸门但照记 OCR/嵌入用量）→ 按类型分派解析引擎 → 产出 `parsed.md + _images/` → `documents` 状态 ready。**解析失败 = 后续一切任务不可发**，前端在文档 ready 前禁用发起按钮。

---

11. 经验自学习闭环

设计文档：审查经验自学习方案.md（v3.0，双层数据模型借鉴 EverOS）/ 经验消化管道开发计划.md。代码：`backend/experience/`。

11.1 数据模型（PG 四表）

- **experience_cases**（微观案例）：task_intent / approach / key_insight / quality_score（三分量）/ finding_ids / 压缩步数比——AgentStep 轨迹的 LLM 压缩产物。
- **experience_skills**（宏观 SOP）：content 为 `## Steps`（已验证）或 `## Potential Steps` + `## Pitfalls`（假设性）；confidence、maturity_score、source_case_ids（≤9）、retired_at。
- **experience_cluster_membership**：case↔cluster（assigned_by=embedding/llm）。
- **experience_feedback**：confirm / contradict / refine + 审批管线。

11.2 闭环数据流

```
【事后·消化】任务终态 → extract_experience.delay → ExperienceService:
   LLM 过滤(值得提取吗) → LLM 压缩轨迹→Case
   (quality_score = 0.6×QualityEvaluator + 0.4×LLM 主观分)
   → 聚簇(embedding 召回[现为桩,全走 LLM] → LLM 从 top-5 判定归属/新建)
   → Skill 双模提取: quality≥0.5 走成功 prompt 产出/更新 ## Steps
                    否则走失败 prompt 产出 ## Potential Steps + ## Pitfalls
   → 成熟度评分(四轴 completeness/executability/evidence/clarity /20,
     带成本门控: 词集变化比<0.2 跳过重评)
   每步 try/except 吞错—绝不阻塞、绝不弄脏主流程
【事前·注入】下次审查 BidReviewAgent.run_review:
   ExperienceRetriever.retrieve(group_id=规则文档名, query=规则前500字)
     SQL(maturity≥0.6 ∧ confidence≥0.1, 按 maturity×confidence 排序)
     ∪ RAG 召回 → LLM 相关性校验(≥0.6) → 按 maturity×confidence×relevance 排序
   → top-3 渲染进系统提示词 {experience_guidance}（hypothesis 标注"来自失败案例，谨慎参考"）
【反馈·演化】用户 confirm/contradict/refine:
   置信度锚定: 新增=0.5, confirm +0.05, 新分支 +0.1, contradict -0.2, cap 0.95
   低于 0.1 → retired_at 退休; refine → LLM 重写 SOP + 条件性重评成熟度
   假设晋升: 现有 skill 含 ## Potential Steps 而新成功案例含 ## Steps
             → 重写为 verified, confidence=0.6, 记 last_promoted_at
```

租户策略是"**全局共享 + 贡献者视图**"：`group_id = 规则文档名` 是硬隔离边界（A001 规则的经验全平台共享），user_id 只用于"我贡献的经验"与审计。总开关 `experience_injection_enabled`（**默认 False**，灰度用），各环节静默降级。

---

12. Agent 工程模式清单（事故沉淀）

每条模式背后是一次生产事故或 ADR。AI Agent 工程师在做设计决策时，先查这张表。

#	模式	防的问题	位置
1	**确定性优先，LLM 收口**	把可正则/可代码的判断交给 LLM → 不可复现、假阳性	blind_check、report_agent、duplicate
2	**解析不出就跳过+待确认，绝不落默认值**	工具参数硬编码样例值与用户实际要求错配 → 大面积假阳性（09-12）	`_build_deterministic_arguments`
3	**回显校验**：执行端回传 expected 与意图比对，错配即弃	旧版客户端用内置默认值执行 → 假违规	`_apply_echo_verification`、broker 回包校验
4	**AI 是用户可见发现的唯一作者**：工具原始违规绝不直出	原始数据脏/口径乱直接吓用户（ADR-0002）	守门员重问 + normalize
5	**分块扫描 + 进度心跳续期 + 按规模自适应超时**	固定超时 × 串行队列雪崩（09-17，466 页文档 21 调用 5 成功）	ChunkedVstoTool、ADR-0003
6	**成对时间戳 + 单调时钟**：下发 requested_at+expires_at，客户端 Stopwatch 判活	用户机时钟偏差 → 整体误判	vsto broker、暗标心跳
7	**异常即工具结果**：工具崩溃转失败 ToolResult，循环不中断	Agent 因单工具故障整任务报废	Mini-Agent run()
8	**软失败工具**：404/超时返回带建议的 success	检索服务抖动打断审查主链	rag_search 等
9	**结果抽取兜底链**：结构化解析→LLM 抽取→关键词扫描	模型输出格式漂移丢全部结果	`_post_process`
10	**防重复注入 user 提醒**（同签名≥3 次注入 [系统提醒]）	Agent 死循环同一工具	`_check_duplicate_actions`
11	**Outbox + 认领锁 + 幂等结算**	发 MQ 与落库的窗口丢任务；重复消费；重复扣费	task_lifecycle、billing
12	**usage 字段风格必须按 provider 实测**：TokenHub 是 OpenAI 风格 `prompt_tokens_details.cached_tokens`，不返回 DeepSeek 风格字段；解析错则兜底全按未命中计价（30 倍价差，08-26 计费事故）	换 provider 静默多扣钱	cost_calculator
13	**只对 success 计费 + flush 闸门**：流水没落齐绝不结算，不产生 0 成本记录	计费口径漂移	finalize_task_usage
14	**NullPool 双引擎**	Celery 每任务新事件循环，池化连接 "Event loop is closed"	models/base.py
15	**新 Celery 任务必须补 task_routes**	消息滞留默认队列且零报错	celery_app.py

#	模式	防的问题	位置
16	**新表 DDL 双审计列 + 引用列全量比对**：`created_at/updated_at TIMESTAMPTZ NOT NULL DEFAULT now()`，发布前对生产 information_schema 比对 ORM 引用的每一列（预发布 create_all 会掩盖 schema 缺失）	UndefinedColumn 500（三踩）	migrations/
17	**max_steps 耗尽 fail-fast，行为异常才重试**	重试放大雪崩	master_agent
18	**LLM 只判定，不改证据**：模型输出绑回不可变候选（相似度/原文/位置）	模型幻觉污染证据链	`_materialize_findings`
19	**输出词表管控**：正则拦截"串标/围标"法律结论	LLM 越权下法律判断	duplicate_check_agent
20	**JSON 容错链**：宽容解析→纠正重试→截断救援	长 JSON 被 max_tokens 截断全废	WizardLLM.generate_json
21	**证据预算分段**（证据≤60%、LLM 收尾预留 150s）	采集吃光预算，结论草草	blind_check 预算

---

13. 扩展开发指南

13.1 新增一个 Agent 工具

1. 在 `backend/agent/tools/` 建类：继承 `mini_agent.tools.base.Tool`，实现 `name/description/parameters`（JSON Schema，中英文参数名双兼容可参考 doc_search.py）与 `async execute()`；返回 backend `ToolResult`，**content 给 LLM 读，data 给代码用。**
2. 遵守软失败约定：任何异常/空结果返回带下一步建议的 success 或明确 error，不让循环中断。
3. 注册：构造 Agent 时加入 tools 列表（或运行时 `self.tools[name] = tool` 动态注入）。
4. 若消耗外部计费资源（OCR/嵌入），在 success/error/timeout **各出口**调用 `record_*_usage`（参考 baidu_ocr.py）。

13.2 新增 Celery 任务（checklist）

- [ ] `backend/celery_app.py` 的 `task_routes` 补路由（review/parser/generation 三选一）——**漏配零报错。**
- [ ] 需要 DB 的任务：每任务新建 session factory（参考 experience_tasks.py），别跨任务复用引擎连接。
- [ ] 长任务配 `task_annotations` 分级 soft/hard 限时。
- [ ] 异步内核用 `asyncio.run()` 包裹；任务体最外层 try/except 保证终态落库。

13.3 接入可计费任务三段式

照抄现有任一任务（最简范本 polish）：API 层 `authorize_billable_task_start` → 同事务建任务行 + outbox → worker `claim_task_for_execution` → 入口 `set_usage_context` → LLM 客户端过 `instrument_llm_client()` → 终态 `finalize_task_usage` → `enqueue_billing_settlement`。**不要自建旁路计费。**

13.4 新增表/列（迁移铁律）

- 新表 DDL 照 `migrations/040` 惯例：uuid 主键 + `created_at/updated_at TIMESTAMPTZ NOT NULL DEFAULT now()`；改表用幂等 `ADD COLUMN IF NOT EXISTS`。
- **ORM/原生 SQL 引用的每一列都必须有对应迁移文件**——预发布由 `create_all` 建表会掩盖缺失，到生产才是 500。
- 发布前对生产 `information_schema` 做引用列全量比对。
- 文件在 `backend/migrations/NNN_描述.sql`，手工执行；本机编辑的 SQL 发往 Linux 前**去 CRLF**。

13.5 切换 LLM provider

纯 `.env` 操作（`LLM_PROVIDER` + key/base/model）+ 滚动重启，但必须：① 先跑 `PYTHONPATH=<仓库根> python scripts/test_deepseek.py` 实测；② **验证 usage 字段风格**（DeepSeek 风格 `prompt_cache_hit_tokens` vs OpenAI 风格 `prompt_tokens_details.cached_tokens`），解析错 = 计费 30 倍价差事故；③ 留意 key 用量余量（旧账号曾 402 断供）。生产滚动重启按集群手册三铁律。

13.6 本地开发环境

用 `scripts/bjt.ps1`（Windows）/ 参照 运行操作指导.md。三个前置坑：① `git submodule update --init Mini-Agent`；② `mkdir workspace`；③ 本地无 PG/Redis 时 `.env` 指内网预发布（需 VPN）。起来标志：`curl :8000/health` → `{"status":"healthy"}`。登录依赖本地/可达的 operate-two（`OPERATE_API_BASE_URL=http://127.0.0.1:40000`，不带 `/prod-api`），或开 `MOCK_AUTH_ENABLED` 旁路。杀本地 `--reload` 残留必须连 `spawn_main` 子进程一起杀。

---

14. 附录：术语表与文档索引

术语表

术语	含义
招标文件 / 投标文件	tender / bid；doc_type 两态
规则库	`docs/rules-all/` 等目录下的审查规则 markdown，审查扇出的单位
暗标	无标识要求的评标形式，检查其格式符合性
TodoItem	一次审查任务内的子任务单位（= 一个规则文档），进度条颗粒
outbox	TaskDispatchOutbox 表，任务可靠派发凭证
点数 / credit lot	计费单位；批次分 recharge（充值）/ gift（赠送），扣减先 recharge 后 gift
interior_user	运营台内部员工标记，决定管理页面可见性
snapshot_id	VSTO 会话绑定的文档快照，防中途换文档
记忆化	`_summarize_messages` 对执行历史的 LLM 压缩
物化	`_materialize_findings`：LLM 决策绑回不可变证据

文档索引（延伸阅读）

主题	文档
需求与数据流/时序图	项目需求.md §2
集群权威实况 / day-2 运维	集群部署方案.md §3.9 / 集群维护速查手册.md
Mini-Agent 交互细节	mini-agent-interaction.md
经验自学习设计/实施	审查经验自学习方案.md / 经验消化管道开发计划.md
查重设计与规划	技术投标文件查重功能-需求设计与开发计划.md 及第二阶段规划
数据库	database_schema.md（⚠ 缺计费两表）
ADR（暗标系列决策）	sales 工作树 `E:\dvlp\dev\sales\docs\adr\`：0001 编标逐章撰写 / 0002 AI 作者口径 / 0003 分块+心跳+预算
部署历史/事故全记录	sales 工作树 `doc/workspace/11-deploy-log.md`、`05-production-env.md`

> **维护约定**：本文随架构演进更新；重大结构调整（新增 Agent 形态、换队列拓扑、换 LLM 协议）必须同步改 §5/§8/§12。凭证类信息勿写本文。